

Divisible Load Theory Retrodicts the Yeco Campaign: The Weights We Guessed, the Optimum the Stealer Found, and the Term We Deleted

Matias Saavedra

Thursday 16th July, 2026

Analysis report — no new renderer measurements. Every input number comes from report 27’s data (experiments/25-frame-distribution/, data commit f310656, code state c5bd3ed). The solver is **DLTlib** (G. Barlas, GPL-3.0, §11.4 of the course text), an *external* sibling directory (Chapter_11_Loadbalancing/DLTlib); the driver, build script, solver output and sweep data live in experiments/28-dlt-analysis/.

Resumen

Reports 26–27 set the static frame shares from node compute speed alone (solo walls, $21.02/5.15 = 4.08 \rightarrow$ weights 1:4) and then repaired the consequences empirically. This report feeds the same measured numbers into the machinery chapter 11 of the course text builds for exactly this problem — collection-aware divisible load theory, via the book’s own DLTlib solver — and checks the theory against everything the campaign measured. It retrodicts all of it. **With the communication term deleted ($l = 0$) the closed form reproduces our guessed weights ($80.3/19.7 \approx 1:4$); with the measured per-file scp cost inserted ($l = 83.5$ ms/frame) the optimum shifts to $60.9/39.1$ and the model prices our 1:4 partition at 11.9 s — the collection wall of report 27, predicted.** The work-stealer’s “backwards” scp equilibrium (42/58) lands within 3 frames of the DLT optimum (39/61): lane-time self-scheduling *is* the model’s finish-time equalization, found by iteration instead of algebra. Post-batching ($l = 12.2$ ms) the optimum is $76.7/23.3$ and our 1:4 sits 0.14 s (2%) from it — inside $\sigma = 0.26$, so no re-weighting is warranted. The model also ranks the levers the way report 27 concluded: perfectly re-weighted scp (9.4 s) still loses to naively-weighted tar (6.2 s) — transport beats scheduling in closed form. Limits: the load is linear only *on average*, and it is the seeded random interleave of DIST_WEIGHTS that makes it so.

1. Motivation: we optimized two of the three terms

Every static share this project has assigned was derived from compute speed alone. Report 26 set laptop:ivy at 5:1 from per-frame throughput; report 27 set laptop:yeco at 1:4 from the solo walls. Both campaigns then hit communication-shaped problems — the unstealable in-flight chunk, the exposed collection wall — and fixed them empirically (the $KCAP \cdot w_r$ cap, tar-stream batching). Chapter 11 of the course text derives the static-partitioning answer *with* the communication terms: an affine

cost model per node (compute $p_i(part_i L + e_i)$, distribution $l_i(a part_i L + b)$, collection $l_i(c part_i L + d)$; Eqs. 11.5/11.7/11.8), closed-form optima whose shares carry the link cost in the denominator ($part_i \propto 1/(l_i(a+c) + p_i)$, Eq. 11.10), sequencing theorems for 1-port platforms, and a library — DLTlib — that evaluates all of it. The question this report answers: *had we used the book’s model from the start, what would it have prescribed, and does it explain what we measured?* It is the closing bracket of the campaign: the dynamic load-balancing case study (this codebase is $\mathcal{S}11.5.1$ of the same chapter) rediscovering the static-LB half of its own chapter from the other direction.

2. Model and calibration

2.1. Mapping the frame distribution onto the DLT cost model

The two-level design maps directly onto a single-level DLT star: the load $L = 100$ frames is arbitrarily divisible at frame granularity, the laptop is the load-originating root that also computes, yeco is a worker behind one link. Three observations fix the model constants:

- **Distribution is free at the margin** ($a = 0$). Every node reads the same ~ 200 -byte `spec.in` and derives its frames from the ownership predicate; shipping the spec is a per-run constant, not a per-frame cost. (It is folded into the 1.2 s fixed orchestration term below, together with the GNU parallel spin-up and per-rank `mkdir/cleanup` round trips measured in report 27.)
- **Collection is proportional** ($c = 1$, l = per-frame PNG cost). Output volume scales with the share: ~ 0.46 MB/frame. The laptop’s own frames end as a local `cp`, so its link cost is 0 — the book makes the same $l_0 = 0$ assumption for the master (p. 903).
- **Node speed is the solo wall over 100** — $p_\ell = 0.2102$, $p_y = 0.0515$ s/frame — exactly the book’s prescription of calibrating p by regression on a timing run ($\mathcal{S}11.5.2$ does it for AES encryption; our “regression” has one point per node because cost is linear in frames by construction).

The link parameter is the book’s ping-pong measurement in our clothes: report 27’s transfer micro-A/B on the identical 79-file/36 MB payload gives $l_{\text{scp}} = 6.60/79 = 83.5$ ms/frame and $l_{\text{tar}} = 0.96/79 = 12.2$ ms/frame. That l changed by $6.9\times$ when only the *protocol* changed is the campaign’s central fact restated in model terms: l belongs to the (path, protocol) pair, not to the wire.

2.2. The solver

DLTlib’s `Network::SolveImage(L, a, c)` implements the collection-aware “image processing” problem class — distribution $a \cdot l \cdot part_i L$, collection $c \cdot l \cdot part_i L$ — which is our shape exactly (the `ImageQuery` family models *constant-size* collection, the wrong shape for PNG streams). The driver builds the star, solves at each transport’s l , and also sweeps $l \in [0, 100]$ ms in 1 ms steps for Fig. 1. One scope limit: `ImageAggregate` reads a single link speed per parent (`temp->link[0]`), so the mixed WAN+LAN

three-node star is evaluated with the hand closed form the 2-node solves validate ($n_i \propto 1/(p_i + l_i)$, $l_\ell = 0$).

Código 1: experiments/28-dlt-analysis/mandel_dlt.cpp — the 2-node solve. DLTlib is unity-included, DLTdemo-style; run.sh builds it against a patched copy of the external library.

```

1 Network net;
2 net.InsertNode((char *)"laptop", P_LAPTOP, 0, (char *)NULL, 0, true);
3 net.InsertNode((char *)"yeco", P_YECO, 0, (char *)"laptop", link, true);
4 net.SolveImage(L, 0.0, 1.0); // a = 0 (spec is free), c = 1 (PNGs)
5 double part0 = net.head->part; // laptop's share
6 double part1 = net.head->child[0]->part; // yeco's share
7 double wall = P_LAPTOP * part0 * L; // root computes start-to-finish

```

Fixed partitions (our DIST_WEIGHTS 1:4) are evaluated under the same cost model as $\max(p_\ell n_\ell, (p_y + l) n_y)$; all predictions add the 1.2 s fixed orchestration constant measured in report 27. Measured comparisons use the report-27 CSVs (results_dynamic_yeco_{_scp,}.csv).

3. Results

Tabla 1: Solver output (results.txt) against the report-27 measurements. “Shares” are laptop/yeco frames of 100. Predicted e2e = compute wall + 1.2 s fixed; “lanes” shows each node’s busy time (laptop compute | yeco compute+collection) — the DLT optimum equalizes them by construction, a fixed partition leaves them unequal. Rows without a measurement are prescriptions, not observations.

Scenario	Shares L/Y	Lanes (s)	Pred. e2e (s)	Measured (s)
DLT optimum, $l = 0$	19.7/80.3	4.14 4.14	5.34	—
DLT optimum, scp	39.1/60.9	8.22 8.22	9.42	—
DLT optimum, tar	23.3/76.7	4.89 4.89	6.09	—
our 1:4 under scp	21/79	4.41 10.67	11.87	13.11 ± 1.81
our 1:4 under tar	21/79	4.41 5.03	6.23	6.45 ± 0.26
dyn steal under scp	42/58	8.83 7.83	10.03	12.95 ± 3.45
DLT scp optimum, rounded	39/61	8.20 8.24	9.44	—
3-node + ivy, tar	22.1/73.0 (+4.8 ivy)	equal	5.85	—

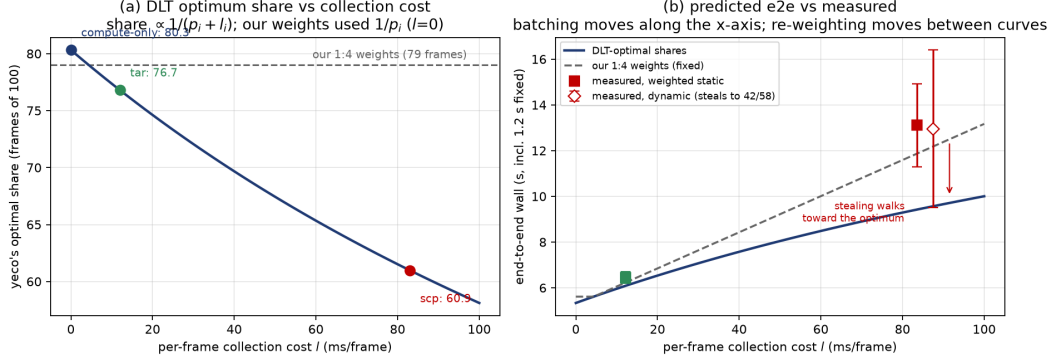


Figure 1: The model across the whole transport axis. Panel (a): the optimal yeco share falls as collection gets more expensive — share $\propto 1/(p_i + l_i)$; the grey dashed line is our fixed 1:4 weights, i.e., the $l = 0$ answer. Panel (b): predicted e2e for DLT-optimal (blue) vs. our fixed 1:4 shares (grey dashed), measured weighted-static points as squares (green tar, red scp) and the dynamic scp run as the open diamond. Batching moves *along* the x-axis (red $\rightarrow$ green); re-weighting only moves *between* the curves — and the gap between curves at tar (0.14 s) is inside measurement noise, while the x-axis move was worth -51% .

4. Analysis

4.1. Setting $l = 0$ reproduces the guessed weights: we ran DLT with the communication term deleted

The compute-only optimum is 80.3/19.7 (Table 1, row 1) — our 1:4 weights to within rounding. This is not a coincidence to admire: with $l = 0$ the closed form degenerates to $part_i \propto 1/p_i$, which *is* the solo-wall ratio rule we used. The implication runs the other way: every consequence the campaign then measured — the 13.11 s collection wall, the $\sigma = 1.81$ WAN jitter, the stealer “rescuing” the schedule — is the model’s predicted cost of the deleted term, $l \cdot part_y \cdot L = 0.0835 \times 79 = 6.6$ s of exposed shipping on a 4.4 s compute lane. Report 27 discovered empirically that “balanced compute exposes collection”; Eq. 11.8 had carried that term all along.

4.2. The model prices the levers the way report 27 concluded: transport beats scheduling

Under scp the DLT optimum is 39.1/60.9 at a predicted 9.42 s — re-weighting alone would have recovered 2.4 s of our 11.87 s. Batching alone (our naive 1:4 under tar) predicts 6.23 s. **The perfectly re-weighted scp schedule loses to the naively weighted tar schedule by 3.2 s:** the reason is structural, visible in panel (b) of Fig. 1 — under expensive shipping the optimum *refuses* the fast node’s capacity (its share falls to 61), so the platform’s aggregate rate drops; cutting l restores the capacity instead of rationing it. This is report 27’s “the actionable lever is collection batching, not scheduling” derived rather than measured — and it generalizes: on this pair, one round of protocol engineering was worth more than any amount of

scheduling optimality.

4.3. The work-stealer converged to the DLT optimum it had never heard of

Report 27 explained the scp-era steal inversion — the *slow* laptop stealing 22 frames from a $4\times$ faster node — as lane-time self-scheduling: each chunk’s shipping sits inside the driver’s dispatch loop, so the empty-bag rule equalizes compute+shipping. The DLT solve makes that explanation exact: finish-time equalization over lanes of rate p_ℓ and $p_y + l$ is precisely the model’s optimality condition, and its scp answer is 39/61 — the coordinator landed at 42/58, within 3 frames, by iteration instead of algebra. The modeled walls agree to 0.6 s (10.03 predicted at 42/58 vs. 9.44 at the optimum; measured 12.95 ± 3.45 , the gap being WAN jitter and per-chunk overhead the static model does not carry). The implication: the dynamic dispenser is not a rival to the static theory — it is a runtime *estimator* of the same optimum, worth its +21% chunking premium exactly when the model’s inputs (p_i, l_i) are unknown or wrong.

4.4. Post-batching, the guessed weights are 2% from optimal — re-weighting is not warranted

At $l = 12.2$ ms the optimum shifts to 76.7/23.3 and predicts 6.09 s; our 1:4 evaluates to 6.23 s. The 0.14 s gap is 2% of the wall and half of the measured $\sigma = 0.26$ s — unresolvable by a 3-rep benchmark, and smaller than the day-to-day thermal drift of the laptop baseline. This closes the calibration question report 27 left open: after the transport fix, the solo-wall weights are effectively optimal, and the collection term, while real, is now too small to justify a second calibration knob. The general rule falls out of panel (a): the fixed-weights error grows with l , so *compute-only weights are safe exactly when the transport is batched*.

4.5. The three-node outlook: ivy is worth 4.8 frames and 0.24 s — capacity, not latency, in closed form

Adding ivy ($p = 0.958$ s/frame, LAN link $l \approx 4$ ms) to the tar-era pair, the lane-equalization optimum hands it 4.8 of 100 frames and shortens the predicted compute wall from 4.89 to 4.65 s — -0.24 s, smaller than the measured σ of the 2-node config it would improve. The model thus reproduces the campaign’s thrice-realized ceiling as arithmetic: a node contributes rate $1/(p_i + l_i)$, ivy’s is $1.04 \text{ s}^{-1}\cdot\text{frames}$ against the pair’s 20.5, i.e., 4.8% more capacity at the price of a third machine, its orchestration, and its failure modes. The book’s negative-*part_i* remark (p. 903: nodes whose share solves negative should be dropped — “finding the optimum set of nodes”) is the same judgment in the limit; ivy’s share is positive but its *value* is sub-noise. DLT answers “should this node join?” before any benchmark is run.

4.6. Validity: the load is linear only on average — and it is the random interleave that makes it so

DLT’s premise is an arbitrarily divisible load with uniform per-unit cost. Our frames violate it individually: the zoom’s cost trend puts $\sim 70\%$ of the work in the deep half

(reports 25/27’s block-orientation effect, a 3.4 s swing between “block” and “block reversed”). The model applies anyway because `DIST_WEIGHTS`’ seeded weighted-random assignment samples that trend uniformly — every node’s *expected* per-frame cost is the global mean, which is exactly the p_i the solo walls calibrate. Contiguous block assignment breaks the premise and would need per-frame cost profiles; the interleave is what buys the right to use four constants instead. Remaining model debts: the 1.2 s orchestration constant is measured, not derived; per-chunk overheads of the dynamic mode (CUDA context, dispatch round trips) are outside the static model; and the 1-port sequencing theory (Theorem 11.3.3, sort by $b_i(p_i+l)$) is vacuous at one remote link but binds the moment a second remote node shares the laptop’s uplink.

5. Recommendations

1. **Set static weights by $1/(p_i + l_i)$, not $1/p_i$.** Same measurement effort (one solo wall + one transfer A/B per node), strictly more correct. On the batched pair it is worth only 0.14 s (2%, sub- σ); on any higher-latency transport the gap grows like panel (b)’s curve spread — 2.4 s at scp’s l .
2. **Run the closed form as a pre-flight before adding a node.** Ten lines of arithmetic ($n_i \propto 1/(p_i + l_i)$, wall = $n_0 p_0 + \text{fixed}$) predicts the achievable e2e and each node’s marginal value before any distributed run: it prices ivy at -0.24 s (skip) and would have priced the entire report-25 equal-share sweep as strictly dominated. The driver in `experiments/28-dlt-analysis/` is the reusable template.
3. **Recalibrate l on any transport change; keep p on any executor-mix change.** Report 26 showed weights belong to the (machine, executor-mix) pair; this report shows the $6.9\times$ protocol dependence of l . The triple of report 27’s recommendation 3 is now two independent knobs with separate cheap calibrations.
4. **Treat the dynamic dispenser as the estimator of this model’s inputs** (S4.3) — its planned rate-adaptive chunk caps are precisely a runtime fit of $1/(p_i + l_i)$ per node, the book’s two-phase “runtime model calculation” idea (S11.3.1). When the fit is already known, weighted static wins by the +21% chunking premium.

6. Conclusion

Fed the campaign’s own measured constants, the chapter-11 model retrodicts every headline of reports 25–27: our guessed 1:4 weights are its $l = 0$ solution (80.3/19.7); the collection wall is the deleted l -term’s predicted 6.6 s; the stealer’s “backwards” 42/58 equilibrium is 3 frames from the closed-form 39/61 optimum; the batched-era 1:4 sits 2% (sub- σ) from optimal, so the empirical calibration stands; and batching dominates re-weighting (6.23 vs. 9.42 s) because a cheaper link restores capacity that an optimal schedule can only ration. The model also prices the next decision for free — ivy joining the batched pair buys 0.24 s, below noise — and its premise

(uniform per-unit cost) is satisfied not by the workload but by our own seeded random interleave, which is why four constants suffice. What the theory does not carry — per-chunk CUDA contexts, WAN jitter, the 1.2 s orchestration floor — is exactly the list of residuals reports 26–27 measured. The campaign closes as a loop: the codebase is the chapter’s dynamic-LB case study, and its distribution layer, tuned empirically, arrived where the same chapter’s static-LB theory had been pointing all along.